

Structures and Union

Structure

賜 Structure is a method of packing different data types to organize the data in a more meaningful way

賜 It is a **derived data type** which can represent several (heterogeneous) data types in a single unit

賜The elements of a structure are always stored in contiguous memory locations.

```
struct tag_name
{
    datatype member_1;
    datatype member_2;
    :
    datatype member_m;
}variable_1, variable_2, ..., variable_n;
```

Example of structure

```
struct student
{
    int Roll_No;
    char Name[25];
    float percent;
} stud_1, stud_2;
```

```
struct time
{
    int second;
    int minutes;
    int hour;
} start_time, end_time;
```

```
struct account
{
    int account_no;
    char
    account_type;
    char Name[25];
    float balance;
}
OldCustmr, NewCustmr;
```

```
struct book
{
    char title[100];
    char author[30];
    int pages;
    float price;
} book[5];
```

Example of structure

```
struct Rectangle
{
int length;
int breadth;
}rect1;
```

Structure Declaration

The tag name can be declared with the structure definition or even later in the main function. Example

```
struct student
{
int Roll_No;
```

```
        char Name[25];
        float percent;
    };
int main() {
struct student stud_1, stud_2;
```

Structure - Initialization

```
struct patient
{
    char Name[25];
    float weight;
    float height;
} p1, p2, p3;
struct patient p1
={ "Ram", 68.5, 172.32}; struct patient p2
```

```
= { "Ima", 65.2, 165.5 } ;
```

Accessing members of a structure

Ex 1 : To access - Name of student

```
stud_1 . Name
```

Ex 3 : To access – Account type of new customer

```
NewCustmr . account_type
```

Ex 4 : To access- Title of Book1

```
book[1] . title
```

The '●' Dot operator establishes the link between member and variable

Input, output of structure member

```
scanf( "%s", stud_1.name) ;  
scanf( "%f",  
&book_2.price) ;
```

```
printf( "%s", account.name) ;  
printf( "%d",  
end_time.hour) ;
```

Assign values

```
strcpy( book[1].title, "Basic
```

```
ME"); book[1].pages = 250;  
book[2].price = 120.50;
```

Operations on members

```
sum = stud_1.marks +  
      stud_2.marks stud_1.Roll_No + 1
```

WAP to declare a structure that contains employee name, date of joining and salary, reads the above information and prints the same.

```
#include<stdio.h>  
struct employee  
{  
    char name[20];  
    int day;
```



```
char month[10];  
int year;  
float salary;  
} e1,e2;
```

Structure – program contd

```
void main()  
{  
    printf("Enter the Name of e1:  
    \n"); scanf("%s", e1.name);  
    printf("Enter the date of joining of e1: \n");  
    scanf("%d %s  
    %d",&e1.day,e1.month,&e1.year);  
    printf("Enter the salary of e1: \n");  
    scanf("%f", &e1.salary);  
    printf("The details of e1 is :%s %d %s %d %f",
```

```
e1.name, e1.day, e1.month, e1.year, e1.salary); }
```

Output

Enter the Name of e1:

Amith

Enter the date of joining of e1:

5
May

2020

Enter the salary of e1:

45000.00

The details of e1 is : Amith 5 May 2020 45000.0000

Array within structures

- ▶ A structure may contain elements of different data types - `int`, `char`, `float`, `double`, etc.
- ▶ It may also contain an **array** as its member. Such an array is called an array within a structure.

Array within structures

```
struct candidate
{
int roll_no;
char      } c1;
grade;
float      Array
marks[3];
```

Initialization

```
struct candidate c1 = {1, 'A', {98.5, 77, 89}};
```

WAP to accept a structure with array member and display it

```
#include<stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks[3];
}s1;
int main()
{
    int i;
    printf("Enter the Roll No. and Name:
    \n"); scanf("%d %s", &s1.roll, s1.name);
    printf("Enter the student marks : \n");
```

WAP to accept a structure with array member and display it

```
for (i = 0 ; i<3; i++)
{
```

```
        scanf("%f", &s1.marks[i]);
    }
    printf("The student details are :\n");
    printf("The Roll No is :
%d\n", s1.roll); printf("The Name is :
%s\n", s1.name); printf("The marks are
:\n");
    for (i=0;i<3;i++)
    {
        printf("%10f", s1.marks[i]);
    }
    return 0;
}
```

Output

Enter the Roll No. and Name :

1

Angel

Enter the student marks :

35

45

45.5

The student details are :

The Roll No is : 1

The Name is : Angel

The marks are : 35.000000 45.000000 45.500000

**WAP to find the sum of marks of a
student using structure**

```
#include<stdio.h>
struct student
{
    int roll;
```

```
char name[50];  
float marks[3];  
}s1;  
int main()  
{  
    int i ;  
    float sum;  
    printf("Enter the Roll No. and Name :  
    \n"); scanf("%d %s", &s1.roll, s1.name);
```

WAP to find the sum of marks of a student using structure

```
printf("Enter the student marks :  
\n"); for (i = 0 ; i<3; i++)  
{  
    scanf("%f", &s1.marks[i]);  
    sum = sum + s1.marks[i];  
}
```



```
printf("The student details are :\n");  
printf("The Roll No is :  
%d\n",s1.roll); printf("The Name is :  
%s\n", s1.name); printf("The sum of  
marks is : %f", sum); return 0;  
}
```

Output

Enter the Roll No. and Name

: ¹Aby

Enter the student marks :

252524

The student details are :

The Roll No is : 1

The Name is : Aby

The sum of marks is : 74.000000

Array of structures

It is an array in which each element is a structure

```
struct student
{
    int roll;
    char name[50];
    float marks;
}s[10];
```

Array of structures

Initialization

```
struct student s[3]  
= { {1, "Ajay", 65.5},  
  {2, "Manju", 79.5},  
  {3, "Bindu", 58.0} };
```

While initialization , empty array can also be written - `s[ ]`

Array of structures

WAP to print the elements of array of structure

```
#include<stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks;
} s[10];
int main()
{
    struct student s[] =
    {{1, "Ajay", 65.5},
    {2, "Manju", 79.5}, {3, "Bindu", 58.0}};
```

**WAP to print the elements of array of
structure**

```
printf("The roll No of first stud is :  
      %d\n",  
      s[0].roll) ;  
  
printf("The name of second stud is :  
      %s\n",  
      s[1].name) ;  
  
printf("The mark of third stud is :  
      %f\n",  
      s[2].marks)  
      ;  
  
return 0;
```

}

Output

The roll No of first stud is : 1 The

name of second stud is : Manju The

mark of third stud is : 58.000000

WAP to accept the input and display the
elements of array of structure

```
#include<stdio.h>  
struct student
```

```

{
    int roll;
    char name[50];
    float marks;
} s[3];
int main()
{
    int i;
    for (i = 0 ; i<3; i++)
    {
        printf("Enter the student[%d] details: \n", i+1);

```

WAP to accept the input and display the elements of array of structure

```

scanf("%d %s %f", &s[i].roll, s[i].name,
    &s[i].marks); }
puts("The student details are
:"); for (i = 0 ; i<3; i++)

```

```
{  
printf("%d %s %f\n",s[i].roll, s[i].name,  
s[i].marks); }  
return 0;  
}
```

Output

45

Enter the student[1] details

¹
: Ann

³⁵
Enter the student[2]

²
details : Milan

⁴⁰
Enter the student[3]

³
details : Karan

The student details are :

1 Ann 35.000000 2

Milan 40.000000 3

Karan 45.000000

WAP to find the sum of marks of 3 students using structure

```
#include<stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks[5];
}s[3];
int main()
{
    int i, j;
    float sum, total[10];
    for (i = 0 ; i<3; i++)
    {
        printf("Enter the student[%d] details : \n", i+1);
        scanf("%d %s", &s[i].roll, s[i].name);
```

WAP to find the sum of marks of 3 students using structure

```

for (j=0; j<5; j++)
    {
        scanf("%f", &s[i].marks[j]);
        sum = sum + s[i].marks[j];
    }
total[i] = sum;
}
printf("The sum of marks of each student is:
") for (i = 0 ; i<3; i++)

{printf("%d %6s %10f\n", s[i].roll, s[i].name,
total[i]); } return 0;
}

```

Output

the student[2] details :

Enter the student[1] details :

2 Milan 35 45.5 45 43.5 44

1 Ann 20 22.5 34 27 40 Enter

Enter the student[3] details :

1 Ann 143.500000 2

3 Karan 34 34.5 45 34 44

Milan 356.500000 3

The sum of marks of
each student is:

Karan 548.000000

Nested structure

A structure within a structure is known as **nested structure in C**.

Syntax

```
struct struct1  
{  
type member1;
```

```
type member2;  
struct struct2 structVar;  
}var1;
```

Nested structure - example

```
struct employee  
{  
    char name[10];  
    float salary;  
    struct dob  
        {  
            int d, m, y;  
        } d1;  
}emp1, emp2;
```

Accessing Members of a Nested Structure

`struct1.struct2.member_name`

Example:

`emp1.d1.d`

employee is the parent structure

Initializing Nested structure

```
#include <stdio.h>
#include <string.h>

struct incomeInLPA {
    int base_salary;
    int ctc;
    struct {
        int bonus;
    };
};
```

```
};                                int employeeid;
    struct                        struct
    employee {                    incomeInLPA inc;
    char name[30];                };

```

Initializing Nested structure

```
int main()
{
    struct incomeInLPA inc = {13, 27, 3};
    struct employee emp = {"Mike", 92,
inc}; return 0;
}

```

or

```
struct employee emp = {"Mike", 92, {13, 27,
3}} or

```

```
struct employee emp ={"Mike", 92, 13,  
27, 3}
```

```
#include <stdio.h>
```

```
struct college
```

```
{
```

```
int college_id;
```

```
char college_name[50];
```

```
}clg;
```

```
struct student
```

```
{
```

```
int id;
```

```
char name[20];
```

```
float percent;
```

```
struct college clg;
```

```
}stud;
```

```
int main()
```

```
{
```

```
struct student stud = {1, "Hezal", 90.5, 13045,
```

```
"CUSAT"}; printf(" Id is: %d \n", stud.id);
```

```
printf(" Name is: %s \n", stud.name);
```

```
printf(" Percentage is: %f \n\n", stud.percent);
```

```
printf(" College Id is: %d \n",stud.clg.college_id);
```

```
printf(" College Name is: %s \n",
```

```
stud.clg.college_name); return 0;
```

}

Output

Id is: 1

Name is: Hezal

Percentage is: 90.500000

College Id is: 13045

College Name is: CUSAT

Structure within structure

WAP to

a. Create a structure with fields:

Name, Address, DOB

b. Read the above details for *two* students from user and display the same

Answer

```
#include<stdio.h>
struct person
{
    char name[50],
    address[200]; struct dob
```

```

    {int dd, mm, yy;
    }date;
}p[2];
int main()
{
    int i;
    for (i = 0 ; i<2; i++)
    {
        printf("Enter the person[%d] details : \n", i+1);
        scanf("%s %s %d %d %d", p[i].name, p[i].address,
        &p[i].date.dd, &p[i].date.mm, &p[i].date.yy);
    }
    printf("The details of persons are :\n"); for (i =
0 ; i<2; i++)
    {
        printf("%s %s %2d.%2d.%4d\n", p[i].name,
            p[i].address, p[i].date.dd,
            p[i].date.mm,
                p[i].date.yy);
    }
    return 0; }

```

Output

Enter the student[1] details :

Sekharan MundakkallHouse; Thrissur 10
10 1990

Enter the student[2] details :

Neelakandan MangalasseryHouse; Kottayam
15 5 1998

The details of students are :

Sekharan MundakkallHouse;Thrissur 10 10 1990

Neelakandan MangalasseryHouse;Kottayam 15 5 1998

Passing structure to function

Two different ways of passing structure •
Passing structure to a function by value •
Passing structure to a function by address
(reference)

1) Structure members can be **individually** passed to a function as arguments in function call 2) **Entire** structure can be passed as using pointer as argument

Passing structure to function

```
#include <stdio.h> #include  
<string.h> struct student
```

```
{  
int id;  
char name[20];  
float percent;  
}record;
```

void struct_print(struct student record); Function declaration

```
int main()  
{  
struct student record; record.id = 1;  
strcpy(record.name, "Raju"); record.percent = 86.5;  
struct_print(record); return 0;  
}
```

Function call

Passing structure to function

```
void struct_print(struct student record)  
{  
    printf("Id is: %d \n", record.id);  
    printf("Name is: %s \n",  
    record.name);  
    printf("Percent is: %f \n",  
    record.percent); }  

```

Function definition

Output

Id is: 1

Name is: Raju

Percentage is: 86.500000

WAP to print the car details using
structure

```

#include <stdio.h>
struct car
{
    char name[30];
    int price;
};

int main()
{
    struct car c = {
        "Tata", 1021 };

    print_car_info(c);

    return 0;
}

void print_car_info(struct car c) {
    printf("Name : %s", c.name);
    printf("\nPrice : %d\n", c.price);
}

```

Output

Name : Tata Price :
1021

Union

- A union allows to store different data types in the

same memory location.

- Union are also derived data types.
- All the members of the union share the same memory location, therefore only one member of the union is can store valid value at one time.
- The compiler allocates enough memory to hold the **largest member** of the union. Since the memory space is shared, this is a way to save the memory.

Union Declaration- Syntax

```
union union_name
{
    datatype member1;
```

```
datatype member2;
```

```
...
```

```
} var_1, var_2, ...var_n;
```

Size of Union

```
union var
```

```
{
```

```
int id;
```

```
char a;
```

```
char b;
```

```
char c;
```

```
char d;
```

```
};
```

- The size of the union is **4 bytes** not **8 bytes**. • All

the members will share the **same memory space**. •

So, the size of the union is determined by the size of the largest member.

- In this case the largest member is **int id**.

Difference between Structure & Union

- The memory occupied by **structure** variable is the **sum of sizes of all the members** but memory occupied by union variable is equal to **space held by the largest data member** of a union.
- In the structure all the members can be accessed at any point of time but in union

only one of union member can be accessed at any given time.

Difference between Structure & Union

```
void main( )
{
#include<stdio.h>  union
job
{
char name[32]; float
salary; int worker_no;
} u;
struct job1
{
char name[32]; float
salary; int worker_no;
} s;

printf("Size of Union =
%d", sizeof(u));
printf("\nSize of
Structure = %d",
sizeof(s));
}
```

Size of Union = **32**

Size of Structure = **40**

Output

Difference between Structure & Union

Structure	Union
Keyword is struct.	Keyword is union.

Size of a structure will get memory equal to the sum of memory of all the members.

Size of a union will get memory equal to the memory of highest member of the union.

All the members can be used simultaneously.

Only one member can store valid value.